

Using Agentic AI for contextualized and multifaceted code review at Ericsson

Muhammad Laiq¹, Ricardo Britto^{1,2}, Muhammad Usman¹, Nishrith Saini²,
and Deepika Badampudi¹

¹ Blekinge Institute of Technology, Sweden.

{muhammad.laiq, muhammad.usman, ricardo.britto, deepika.badampudi}@bth.se

² Ericsson AB, Sweden.

{ricardo.britto, nishrith.saini}@ericsson.com

Abstract. Context: Conducting effective code reviews is increasingly challenging due to the growing complexity of software systems and the accelerated code generation by AI coding agents. LLM-based approaches for code reviews have shown promising results in identifying defects and improving code quality. However, existing approaches rarely consider project-specific contextualized knowledge, and few have been evaluated in industrial settings.

Objective: In this study, we propose a multi-agent-based solution that provides multifaceted assessments of code changes.

Method: Following the Design Science Research Process, we developed and evaluated our solution in an industrial setting. Our solution combines specialized agent skills with context-specific knowledge to identify antipatterns in code changes across four dimensions: readability, maintainability, reliability, and performance. Using our solution, we generated reviews for several code commits and identified more than 200 issues. These issues were then manually validated by the developers of the case company for their correctness and importance.

Results: The evaluation results show that our solution achieves 96% accuracy in correctly identifying issues in the investigated code commits. Furthermore, around 69% of the correctly identified issues were rated as important, with approximately 33% rated as severe issues that must be fixed and 36% as important issues that should be fixed. Qualitative feedback from developers corroborates these findings and highlights the usefulness of the generated reviews.

Conclusion: Our findings provide empirical evidence from an industrial evaluation that combining specialized agent skills with context-specific knowledge yields accurate, practically useful code reviews.

Keywords: Modern code reviews · Agentic code review · Multifaceted code review · Contextualized code review.

1 Introduction

Code review is a fundamental practice in software engineering that helps improve software quality and facilitates knowledge sharing among developers [4, 6, 3]. Developers routinely perform code reviews to assess code changes, for example, to

identify defects, maintain coding standards, and ensure the long-term health of software systems. However, conducting effective reviews remains a challenge. Reviewers are expected to understand the intent of code changes, reason about their impact on the broader system, and identify potential issues, all while working within fast-paced development cycles. This imposes a significant cognitive load on the reviewers. The challenge becomes even greater as software systems continue to grow in scale and complexity, making it increasingly difficult to maintain thorough, consistent, and timely reviews [17, 2, 18]. This challenge is further amplified by the rise of agentic AI in software development. As organizations delegate an increasing share of implementation work to AI coding agents, code is produced at a speed and volume that human reviewers cannot match, making reviewers a bottleneck and increasing their cognitive load. This is particularly acute in companies moving toward a fully AI-native software development approach, where orchestrated agent workforces operate in parallel, and assurance is expected to keep up with code generation rather than lag behind [7].

To assist practitioners in code review, several automated solutions have been proposed in the literature [32, 3]. These solutions include static analysis tools, traditional machine learning and deep learning-based approaches, and more advanced large language models (LLMs). Although prior work has made significant progress and shown positive results, important gaps remain. For example, many of these approaches focused on providing a narrow review [30, 32]. In practice, reviewers rarely assess code changes from a single perspective. Instead, they apply a multifaceted reasoning process that considers a range of software quality concerns simultaneously. For example, maintainability, readability, and other non-functional qualities. Another important challenge concerns the role of contextual knowledge in review activities. Human reviewers rely extensively on contextual information, including system architecture, organizational standards, dependencies, and domain-specific constraints, when assessing the implications of a code change. Previous work also emphasizes that reviewers need both the local and global context to effectively understand and assess changes [30]. However, much of the current work uses limited context-specific knowledge [2, 32, 30]. As a result, many existing approaches struggle to provide feedback that reflects the broader context in which software systems evolve. In addition, evaluations of the proposed approaches in real industrial settings remain relatively limited.

This study aims to contribute towards filling the above-mentioned gaps. We propose a contextualized multi-agent code review approach. The proposed approach combines multiple specialized agents with context-specific knowledge to support a multifaceted assessment of code changes. Within the scope of this study, we assess code changes across four dimensions: readability, maintainability, reliability, and performance. The study follows the Design Science Research Process by Offermann et al. [21]. The solution is developed and evaluated in an industrial context (Ericsson).

In this study, we answered the following research questions.

- **RQ1:** What design choices are essential for providing multifaceted and contextualized code review?

- **RQ2:** How accurate is the proposed approach in identifying issues from code commits?
- **RQ3:** How effective is the proposed approach in terms of identifying important issues from code commits?

We assess accuracy by measuring the proposed approach’s ability to identify valid issues in code changes, with each identified issue labeled correct or incorrect, and effectiveness by measuring the extent to which correctly identified issues are considered worth fixing, with each correct issue rated as minor, medium, or high.

This paper is organized as follows. Section 2 describes the related work on the topic. Sections 3–4 present the research design and the proposed solution. Section 5 presents the results of the study. Section 6 discusses the findings of the study. Section 7 describes the threats to validity. Finally, Section 8 concludes the paper with future work.

2 Related work

There is a plethora of research on code review that spans several dimensions, including automated reviewer assignment, code comment analysis, and code change assessment [32, 3]. Among these areas, the most relevant work for this study is the assessment of code changes to provide review comments.

Significant research has focused on assessing code changes to generate code review comments. Early automated code review approaches primarily relied on static analysis and rule-based techniques. Such approaches focused on identifying predefined patterns, coding standard violations, or structural issues in source code [15, 5]. Although these tools are effective for detecting specific categories of issues, they generally provide limited support for higher-level reasoning about software quality. In practice, these approaches often operate independently of the broader development and architectural context in which code changes occur. Subsequent research explored machine learning and deep learning approaches to automate code review. For example, Gupta et al. [13], Li et al. [19], and Shi et al. [26] used deep learning-based approaches to automate code review. In particular, Convolutional Neural Networks and Long Short-Term Memory were used to build models. Although these approaches have improved automated code review compared to previous methods, their capabilities are largely limited by the quality and scope of training data, and they often struggle to generalize beyond localized patterns.

Recent advances in LLMs have substantially changed the landscape of automated code review research. Due to their ability to process source code and natural language jointly, LLM-based approaches have shown improved performance in code understanding, review comment generation, and reasoning about code changes [10, 22, 8, 32, 23, 28]. For using LLMs in this task, different strategies have been investigated in the literature. For example, Nashaat and Miller [20] fine-tuned pretrained models such as T5 and CodeT5 for code review, while Ramesh et al. [22] investigated prompt-based approaches using two variants of

Llama models without task-specific fine-tuning. Other studies proposed multi-agent and tool-augmented architectures to improve review reasoning [23, 28].

Despite significant progress and promising results from LLM-based approaches to code review, several gaps still exist. For example, a recent survey by Yang et al. [32] found that current approaches often use limited contextual information. Many rely mainly on isolated diffs or local code snippets, while effective reviews often depend on organizational and repository-specific context. Furthermore, they noted that much of the work provides only a narrow review. Only a few studies, such as those by Ren et al. [23] and Sun et al. [28], have moved toward a multidimensional review through taxonomies. Similar gaps have also been reported by Unterkalmsteiner et al. [30]. Finally, we observe that industrial evaluations of LLM-based code review approaches remain relatively limited. Much of the current literature relies on offline experiments or open-source datasets.

Consequently, this study proposes a multifaceted contextualized code review approach. The proposed approach uses multiple specialized agents, each targeted at different aspects of code changes. For example, an agent for assessing code maintainability and an agent for assessing code reliability. In addition, the approach integrates system-specific knowledge via a code knowledge graph and reusable agent skills, enabling agents to reason beyond submitted code diffs. The proposed solution is developed and evaluated in the real industrial setting, providing evidence of its practical applicability beyond offline experiments.

3 Research design

This study employs the Design Science Research Process (DSRP) proposed by Offermann et al. [21] to develop and evaluate technological artifacts. We operationalize DSRP through three phases: problem identification, solution design and development, and evaluation (Sections 3.1–3.3). The solution (resulting artifact) is described in detail in Section 4, and the results of the evaluation are reported in Section 5.

3.1 Phase 1: Problem identification

In this phase, the problem was identified by reviewing the relevant literature (see details in Section 2). Beyond the literature, the problem was discussed with practitioners at the case company to understand current review practices and the challenges reviewers face, particularly as AI coding agents produce an increasing share of code. Three practitioners were involved: one manager and two senior developers. These discussions confirmed the problem’s practical relevance and, together with the literature, informed the solution’s scope. For this initial iteration, it was decided to focus on the four dimensions of the code review and to ground the assessment of each dimension in an established antipattern catalog (see details in Section 4.1).

3.2 Phase 2: Solution design and development

This phase translates the identified problem into a working artifact: a contextualized multi-agent solution that provides a multifaceted assessment of code changes. The artifact is based on two main design decisions. First, the review is decomposed into specialized agents, each dedicated to one quality dimension and guided by its own prompt, antipattern catalog, and review instructions, which addresses the narrow-review gap. Second, the agents do not reason about isolated diffs. A context builder supplies each agent with a contextual view constructed from the code repository and a code knowledge graph that addresses the limited-context gap. The artifact, its components, and the antipattern catalogs used by each agent are described in Section 4.

3.3 Phase 3: Evaluation

The evaluation was designed as a static validation [12], in which the solution is applied in a realistic industrial context (Ericsson). We generated reviews for seven code commits from four different projects at the case company. Two of these projects were small, and two were large. In total, 206 issues were identified from the selected code commits. We then conducted a user study to evaluate their correctness (RQ2) and importance (RQ3). The issues were reviewed by the developers of the code commits, all of whom are senior developers. We used the template shown in Table 1 to collect feedback from the developers on issues identified. In total, five developers reviewed the 206 issues: four of them reviewed one commit each, with 36, 36, 27, and 11 issues, respectively, and one reviewed three commits with 96 issues in total.

Table 1. Feedback template for user study

Part I: Details of an identified issue	
Antipattern:	Antipattern/Name of the identified issue, e.g., The Blob.
Location:	Location of the identified issue, e.g., file name and line numbers.
Description:	Detailed description of the issue.
Fix suggestion:	Fix suggestion.
Part II: User feedback for the identified issue	
A. Correctness of the identified issue	
— Correct	☐
— Incorrect	☐
B. Importance of the identified issue	
— Minor:	Minor or low-impact suggestion ☐
— Medium:	Important issue that should be fixed ☐
— High:	Severe issue that must be fixed ☐
C. Free text field: Optional feedback	

4 The proposed solution

Figure 1 presents an overview of the proposed framework. The framework aims to provide a multifaceted assessment of code changes using specialized code

review agents. Each agent assesses code changes considering repository-specific contextual information. Although the framework can be readily extended to additional code quality aspects, this study focuses on the following four key dimensions, each one with a dedicated AI agent: readability, maintainability, reliability, and performance.

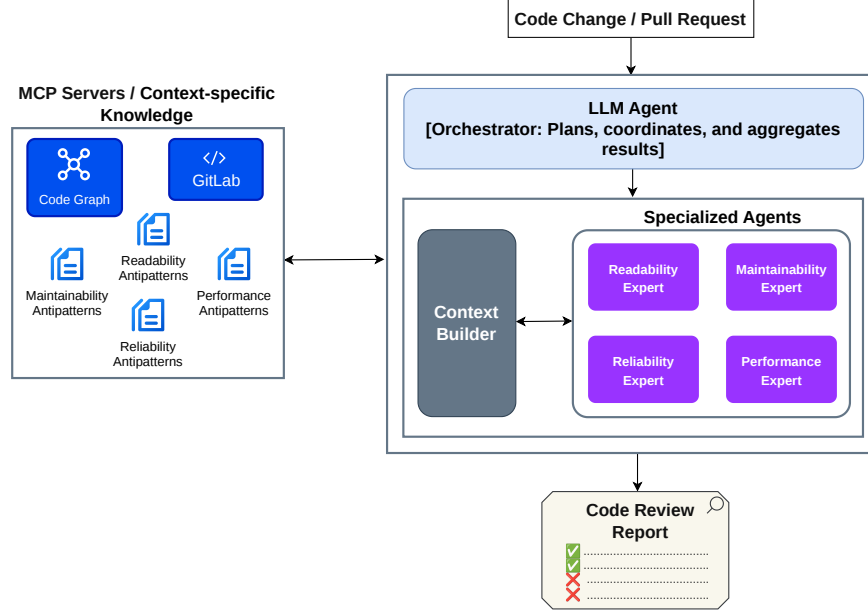


Fig. 1. Overview of the proposed framework

4.1 Code review dimensions focused

A. Readability Expert: The readability agent is focused on assessing whether the modified code is easy to understand and follow. The assessment focuses on several aspects, such as naming clarity, structural organization, nesting complexity, code formatting, visual readability, and the use of understandable programming constructs. To guide the analysis, the agent relies on the readability antipatterns summarized in Table 2. These antipatterns have previously been used in the literature [25, 24] to assess code readability.

B. Maintainability Expert: The maintainability agent focuses on long-term software evolution concerns. In particular, it identifies issues that may increase future maintenance effort, reduce modularity, or complicate code evolution. The agent uses classical maintainability antipatterns [16, 11], including duplicated code, long methods, feature envy, large classes, message chains, and speculative generality, as summarized in Table 3.

Table 2. Antipatterns for readability [25, 24]

Code structure: Logical separation of functionality vs. tangled code.
Nesting: Flat, linear code vs. deeply nested blocks.
Understandable Goal: Clear task or function vs. ambiguous purpose.
Code Length: Concise and readable vs. unnecessarily long code.
Inline Actions: One action per line vs. multiple actions in the same line.
Reading Flow: Code reads well from top to bottom, or while reading, the eyes jump from top to bottom and back up again.
Sufficient Contextual Info: Code is not sufficiently explained and needs additional info to understand what it does, or code is overexplained.
Code Style: Code conforms to style guides, or code is poorly formatted.
Magic Numbers: Code uses named constants or code uses magic numbers.
Naming: Naming clarifies code functionality, or naming is confusing.
Code Patterns: Code uses basic, known code patterns, or code looks unfamiliar or nonstandard.
Visual Organization: There is balance in the color blocks, or there are huge chunks of color blocks that stand out in a distracting way.

Table 3. Antipatterns for maintainability [16, 11]

Mysterious Name: A name that doesn't clearly explain its purpose.
Duplicated Code: The same code structure appears in multiple places.
Long Function: A function that tries to do too much and becomes hard to understand.
Long Parameter List: A function requires too many parameters to operate.
Global Data: Data accessible everywhere, making code fragile and hard to track.
Mutable Data: Data that changes unexpectedly, leading to bugs and side effects.
Divergent Change: One module often needs many modifications for different reasons.
Shotgun Surgery: A single change requires edits across many different classes.
Feature Envy: A method uses another object's data more than its own.
Data Clumps: Groups of data items that always appear together.
Primitive Obsession: Overuse of basic types instead of small, meaningful objects.
Repeated Switches: Multiple conditional statements checking the same conditions.
Loops: Manual loops that obscure intent and could be replaced with higher-level constructs.
Lazy Element: A class or method that no longer justifies its existence.
Speculative Generality: Code created just in case without real need.
Temporary Field: An object with fields used only in certain situations.
Message Chains: Navigation through multiple objects to get data.
Middle Man: A class that delegates all its work to others without adding value.
Insider Trading: Classes overly reliant on each other's internal details.
Large Class: A class doing too much, becoming bloated and complex.
Alternative Classes with Different Interfaces: Classes similar in behavior but exposing inconsistent interfaces.
Data Class: A class with only fields and no meaningful behavior.
Refused Bequest: A subclass that inherits methods or fields it doesn't need.
Comments: Comments are used to excuse unclear/messy code instead of improving it.

C. Reliability Expert: The reliability agent evaluates robustness-related concerns, particularly exception handling or fault management practices. The assessment focuses on identifying antipatterns [9], such as over-catch of exceptions, empty handlers, generic exception propagation, and improper logging strategies. The antipattern catalog used by this agent is presented in Table 4.

D. Performance Expert: The performance agent analyzes code changes from an efficiency and resource utilization perspective. The analysis targets common software performance antipatterns [28, 31, 1, 27], including inefficient database access, excessive I/O operations, repeated calculations, unoptimized loops, inappropriate data structures, and excessive synchronization overhead. The antipatterns considered in this study are summarized in Table 5.

Table 4. Antipatterns for reliability [9]

Over-catch:	The handler catches multiple different lower-level exceptions.
Over-catch and Abort:	Besides over-catching, the handler aborts the system.
Unhandled Exceptions:	The handler does not catch all possible exceptions.
Unreachable Handler:	The handler does not catch any possible exception.
Catch and Do Nothing:	The handler is empty.
Catch and Return Null:	The handler contains return null.
Catch Generic:	The handler catches a generic exception type.
Destructive Wrapping:	The handler propagates the exception as a new exception.
Dummy Handler:	The handler only displays or logs some information.
Ignoring InterruptedException:	The handler catches InterruptedException and ignores it.
Incomplete Implementation:	The handler only contains TODO or FIXME comments.
Log and Return Null:	Besides being a dummy handler, the handler returns null.
Log and Throw:	The handler logs some information and propagates the exception.
Multi-Line Log:	The handler divides log information into multiple log messages.
Nested Try:	The handler and its try block are enclosed in another try block.
Throw within Finally:	The handler is followed by a finally block that propagates exceptions.
Throws Generic:	The throws propagates a generic exception type.
Throws Kitchen Sink:	The throws propagates multiple exceptions.

Table 5. Antipatterns for performance [28, 31, 1, 27]

The Stifle:	Data is retrieved by means of many similar (or equal) database queries.
Expensive Database Call:	A single long-running database request causes performance overhead.
Empty Semi Trucks:	An excessive number of requests is required to perform a task.
The Blob:	A single class/component performs all of the work or holds all of the application’s data.
Circuitous Treasure Hunt:	A high amount of requests to retrieve the data.
Wrong Cache:	Memory pollution through improper use of a cache.
One Lane Bridge:	Mutual access to a shared resource is badly designed.
Inappropriate Data Structures:	Using data structures that are inefficient for the required operations, causing unnecessary performance overhead.
Unoptimized Loops:	Writing loops that perform excessive work or avoid optimizations, leading to slow and wasteful execution.
Data Format Conversion Performance:	Repeatedly converting data between formats in costly ways that degrade runtime efficiency.
Excessive or Improper Lock Usage:	Overusing or misusing locks, creating thread contention, and reducing parallel performance.
Excessive I/O Operations:	Performing more disk or network I/O than necessary, resulting in significant latency and bottlenecks.
Repeated Calculations:	Recomputing values unnecessarily instead of caching or reusing results, wasting CPU cycles.

4.2 Implementation details

The proposed framework uses a multi-agent architecture coordinated through an LLM-based orchestration layer. The prototype is implemented using Kiro CLI³. Kiro CLI is a terminal-based AI coding assistant that supports multi-agent workflows through declarative agent definitions, skill-based behavior specification, and sub-agent delegation. Kiro CLI provides a runtime environment where agents are defined as JSON configurations with associated skill documents (markdown-based behavioral specifications), enabling reproducible and auditable agent interactions without custom code. The system integrates with external data sources via the Model Context Protocol (MCP), an open standard for connecting LLM-based agents to tools and data services. In the following, we describe each component of our framework.

³ <https://kiro.dev/cli/>

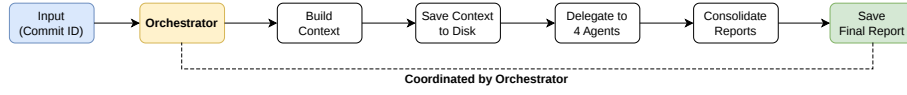


Fig. 2. Code review orchestration workflow

LLM orchestrator: The orchestrator serves as the central coordination component of the framework (see an overview in Figure 2). It is defined declaratively through a JSON agent configuration and a persona document that specifies its behavioral rules. The orchestrator agent configuration includes the agent’s skill, tools, MCP servers, and other relevant settings. When a user prompt (e.g., review the following code commit: #et5253e) is submitted, the orchestrator first analyzes it and determines which specialized agents should participate in the review. Then it invokes the context builder to retrieve the required contextual information and makes the relevant context available to each agent. After the specialized agents complete their analyses, the orchestrator aggregates the output into a unified review report. The report contains a summary of the review, e.g., the number of issues found per dimension, followed by the issues identified for each dimension in the four-field format shown in the first part of Table 1.

MCP servers: In this study, two MCP servers are used: GitLab and Code Knowledge Graph.

(a) GitLab MCP server: A containerized server (deployed via Docker) that exposes GitLab API operations as MCP tools. It provides access to the code repository, for example, to get commit metadata, code diffs, and changed files. This information provides the local context associated with a code change.

(b) Code Knowledge Graph MCP server: A containerized server that exposes a code knowledge graph as an MCP tool. The knowledge graph indexes repository structures, file-level import relationships, symbol definitions (DEFINES edges), and external library usage. It supports the following operations: dependency impact analysis (fan-in/fan-out per file), folder structure exploration, AI-generated file/folder summaries, semantic/hybrid code search across repositories, and direct Cypher query execution.

Context builder: The context builder is responsible for collecting, organizing, and preparing contextual information for agents. For a code change, the context builder retrieves the code diff, modified files, and repository metadata from GitLab. Dependency information and structural relationships are additionally retrieved using the code graph. The information collected is transformed into agent-specific contextual prompts. Different agents receive different contextual views depending on their review objectives. For example, the readability agent focuses on local code structure and formatting, whereas the maintainability agent also receive information about dependencies and structural relationships. This selective context construction aims to reduce prompt complexity while preserving the contextual information most relevant to each review dimension.

Specialized review agents: As shown in Figure 1, we implement four specialized LLM agents: Readability Expert, Maintainability Expert, Reliability Expert, and Performance Expert. Each agent operates independently and is guided through dedicated prompts, antipattern definitions, and review instructions. We use the Agent Skills⁴ protocol for all agents. Each agent is defined by a JSON configuration that specifies its name, system prompt, attached skill document (containing the antipattern checklist), and tool permissions. Agents produce findings in a strict four-field format (Antipattern Name, Location, Problem Description, Fix Suggestion). The agents are invoked as sub-agents by the orchestrator and run in parallel, each receiving only the path to the shared context document and source files on disk.

5 Evaluation results

In this section, we present the evaluation results of the proposed approach.

Table 6. Identified issues per dimension from the studied code commits

Dimension	Count (%)
Reliability	64 (31.1%)
Readability	58 (28.2%)
Maintainability	50 (24.3%)
Performance	34 (16.5%)
Total	206

5.1 RQ1: What design choices are essential for providing multifaceted and contextualized code review?

RQ1 examines which design choices are essential for a multifaceted and contextualized code review, based on observations during solution validation.

Orchestration and output format: All commits were processed through the complete workflow, from agent selection to aggregation into a unified report. The orchestration is a single point of failure, as an error at any stage propagates to the final review, even when the individual agents reason correctly. Validating the context builder and the report format was therefore necessary, and providing agents with positive and negative examples was an effective way to constrain their output.

Specialized review agents: Each agent contributed findings (see Table 6) that the other agents did not, and a review targeting a single dimension would have covered at most about a third of what was reported. Decomposing the review into agents with dedicated prompts, antipattern catalogs, and review instructions is thus what enables multifaceted assessment.

⁴ <https://agentskills.io/home>

Contextual grounding: Only 4% of the identified issues were judged incorrect. We attribute this low rate to the contextual information provided through the context builder and the code knowledge graph. Rather than reasoning over isolated diffs, each agent received the local context and, where relevant, structural and dependency information about the surrounding system, which appears to reduce hallucinated comments. Providing each agent with a contextual view tailored to its dimension, rather than a single undifferentiated context, also reduced prompt complexity.

These observations relate to the design presented in this study. Since neither an ablation nor a comparison with alternative designs was performed, they provide feasibility evidence rather than controlled comparative proof.

5.2 RQ2: How accurate is the proposed approach in identifying issues from code commits?

RQ2 examines whether the issues reported by the proposed approach describe real concerns in the reviewed code rather than false or hallucinated findings.

As shown in Table 7, 197 of the 206 issues were confirmed correct, resulting in an overall accuracy of approximately 96%. Only 9 issues (approximately 4%) were judged incorrect. This low rate of incorrect findings is particularly relevant in an industrial context, where reviewers tend to disengage from automated tools that produce frequent false positives. These results indicate that the approach achieves a level of accuracy that could make it viable for practical use. However, further validation across more diverse projects/contexts, and across programming languages, is needed to strengthen the generalizability of these findings.

Table 7. Correctness of the identified issues

Item	Count (%)
Correct	197/206 (96%)
Incorrect	9/206 (4%)

5.3 RQ3: How effective is the proposed approach in terms of identifying important issues from code commits?

Table 8. Importance of the correctly identified issues (197/206)

Item	Count (%)
Minor — Minor or low-impact suggestion	62 (31%)
Medium — Important issue that should be fixed	70 (36%)
High — Severe issue that must be fixed	65 (33%)

A correct finding is not necessarily a valuable one. An automated reviewer may report accurate but trivial issues that add little value and increase the reviewer’s effort. RQ3, therefore, examines the practical importance of the findings. For this analysis, we consider only the 197 issues confirmed as correct in RQ2 and examine the importance ratings assigned by the developers (reviewers) on a three-level scale: *minor* (low-impact suggestion), *medium* (important issue that should be fixed), and *high* (severe issue that must be fixed).

Table 8 presents the results for RQ3. Among the 197 correct issues, 33% (65) were rated as high importance (issues that must be fixed) and 36% (70) were rated as medium importance (issues that should be fixed). Together, these two categories account for approximately 69% of the correctly identified issues, indicating that the majority of the findings were considered important by the developers and required addressing. The remaining 31% (62) issues were rated as minor or low-impact suggestions. While minor findings may not be critical, they are not without value, as they often relate to readability and stylistic concerns that contribute to the long-term health of the codebase.

In general, the combination of high accuracy (RQ2) and a high proportion of important findings (RQ3) suggests that the approach produces reviews that are reliable and meaningful in practice. In addition, qualitative feedback from the developers corroborated these results. The reviewers were positive and highlighted the usefulness of the generated reviews, for example:

- Comment-1: *"The review comments were really good, especially for the maintainability and performance part."*
- Comment-2: *"A portion of the above comments were almost the same as we had identified but your reviews had more. This was really good."*

6 Discussion

The results of this study indicate that combining specialized agent skills with context-specific knowledge yields accurate, practically relevant code reviews. The approach achieved approximately 96% accuracy in correctly identifying issues in the investigated code commits. Additionally, among the correctly identified issues, approximately 69% were rated as important issues (33% severe, which must be fixed, and 36% important with a medium rating, which should be fixed). The low rate of incorrect findings can be attributed to the contextual grounding described in Section 5. Supplying agents with repository-specific information, rather than code change alone, appears to constrain their reasoning to what actually holds in the surrounding system. Previous work also highlights the role of contextual information in generating accurate code reviews [8, 14, 29, 22]. For example, Cihan et al. [8] investigated the impact of supplementary descriptions (e.g., comments or pull request descriptions) on the accuracy of LLM-based code review. They found that providing such descriptions improved accuracy from 59.62% to 68.50% for GPT-4o and from 55.56% to 63.89% for Gemini.

The low false positive rate is particularly important in practice, as automated reviewers that produce frequent false positives tend to erode developers’ trust,

increase review noise, and ultimately cause developers to disengage from such tools. In addition, the multifaceted assessment proved valuable in identifying concerns that a single-perspective review would have missed. The identified issues were distributed on all four dimensions: reliability (31.1%), readability (28.2%), maintainability (24.3%), and performance (16.5%). This breadth more closely mirrors how human reviewers reason in practice, where multiple quality concerns are considered simultaneously rather than in isolation.

Beyond the empirical results, developing the solution yielded the following takeaways that we believe are broadly relevant to others building agentic code review systems.

- **Orchestration accuracy matters as much as agent design:** An error in agent selection, context construction, or aggregation propagates to the final review regardless of how well the individual agents reason.
- **Output format is part of the design, not a presentation detail:** Constraining findings to a strict format, supported by both positive and negative examples, made a systematic assessment of 206 issues feasible.
- **Separation of concerns should extend beyond the agents:** Giving each agent its own prompt, antipattern catalog, and contextual view reduces prompt complexity and keeps the solution extensible, as covering an additional quality dimension amounts to adding an agent rather than modifying the existing ones.

7 Threats to validity

Construct validity: We study whether the generated reviews are trustworthy and useful to developers in practice. We operationalize trustworthiness as the correctness of each identified issue (RQ2), i.e., whether it describes a real property of the reviewed code rather than a hallucinated one, and usefulness as its importance on a three-level scale (RQ3), i.e., whether a correct finding is worth acting upon. Both measures rely on human judgment, which we consider appropriate for our research questions, since whether a finding is accurate and worth fixing depends on the semantics of the change and on system-specific knowledge.

Internal validity: Having the authors (developers of the code commits) validate the findings in their own code can introduce bias in either direction, that is, greater leniency or greater scrutiny. However, the authors are best positioned to judge the relevance of a finding given their knowledge of the change.

External validity: This study has several limitations that should be considered when interpreting the findings. First, the evaluation was conducted in a single company using seven commits from four projects, and the code studied was limited to Python. Although this provides realistic industrial evidence, the results may not generalize to other organizations or programming languages, and validation in other contexts is needed to assess external validity. Second, the results are based on a single LLM (Kiro CLI’s default model), and the reported accuracy or importance ratings may vary with other models/frameworks. A systematic comparison across models/frameworks covering several aspects such as

accuracy, ease of use, cost, and privacy would be important for organizations weighing adoption and provides a clear direction for future work. In addition, in this study, we have considered only the four dimensions of code quality. Future work may include other aspects, such as security, technical debt, and testability.

Reliability: A threat specific to LLM-based solutions is non-determinism: repeated runs on the same commit may not produce identical findings. To support repeatability, the agents are specified declaratively via JSON configurations and skill documents, and the antipattern catalogs used by each agent are reported in Section 4.1, enabling the setup to be reconstructed. The data collection instrument is also reported in full (Table 1). However, the code commits and the generated reviews are proprietary and cannot be shared, which limits independent replication.

8 Conclusion and future work

In this study, we proposed a contextualized multi-agent-based solution that provides a multifaceted assessment of code changes. The approach combines specialized review agents with context-specific knowledge to identify antipatterns in four dimensions: readability, maintainability, reliability, and performance. We developed and evaluated the solution in an industrial setting. The evaluation results for over 200 issues showed that the proposed solution achieves approximately 96% accuracy in correctly identifying issues. In addition, around 69% of the correctly identified issues were rated as important issues (with 33% considered severe that must be fixed and 36% important issues that should be fixed). Qualitative feedback from the developers corroborated these results and highlighted the usefulness of the generated reviews.

In future work (the next iteration of the design science research process), we plan to evaluate our solution in other contexts, including other programming languages and projects, and to include additional code quality aspects, such as security and technical debt. We also aim to implement feedback mechanisms that enable the system to learn from developers’ feedback. Finally, we plan to extend the solution to support InnerSource projects, in which core teams must ensure that contributions from developers outside the project align with the project’s conventions, architecture, and design decisions.

Acknowledgments. This work was partially supported by the Knowledge Foundation through the InScale project (reference number 20230095) at Blekinge Institute of Technology (BTH), Sweden.

Disclosure of Interests. The authors declare that they have no known competing financial interests or personal relationships that could have influenced the work reported in this paper.

References

1. Avritzer, A., Britto, R., Trubiani, C., Camilli, M., Janes, A., Russo, B., Van Hoorn, A., Heinrich, R., Rapp, M., Henß, J., et al.: Scalability testing automation using

- multivariate characterization and detection of software performance antipatterns. *Journal of Systems and Software* **193**, 111446 (2022)
2. Bacchelli, A., Bird, C.: Expectations, outcomes, and challenges of modern code review. In: 2013 35th international conference on software engineering. pp. 712–721. IEEE (2013)
3. Badampudi, D., Unterkalmsteiner, M., Britto, R.: Modern code reviews—survey of literature and practice. *ACM Transactions on Software Engineering and Methodology* **32**(4), 1–61 (2023)
4. Bavota, G., Russo, B.: Four eyes are better than two: On the impact of code reviews on software quality. In: 2015 IEEE International Conference on Software Maintenance and Evolution. pp. 81–90. IEEE (2015)
5. Beller, M., Bholanath, R., McIntosh, S., Zaidman, A.: Analyzing the state of static analysis: A large-scale evaluation in open source software. In: 2016 IEEE 23rd International Conference on Software Analysis, Evolution, and Reengineering. vol. 1, pp. 470–481. IEEE (2016)
6. Bosu, A., Carver, J.C., Bird, C., Orbeck, J., Chockley, C.: Process aspects and social dynamics of contemporary code review: Insights from open source development and industrial practice at microsoft. *IEEE Transactions on Software Engineering* **43**(1), 56–75 (2016)
7. Britto, R., Palmgren, F., Saini, N., Ohlin, M.: The ai-native large-scale agile software development manifesto. *arXiv preprint arXiv:2605.07717* (2026)
8. Cihan, U., Haratian, V., İçöz, A., Gül, M.K., Devran, Ö., Bayendur, E.F., Uçar, B.M., Tüzün, E.: Automated code review in practice. In: 2025 IEEE/ACM 47th International Conference on Software Engineering: Software Engineering in Practice. pp. 425–436. IEEE (2025)
9. De Padua, G.B., Shang, W.: Studying the prevalence of exception handling antipatterns. In: 2017 IEEE/ACM 25th International Conference on Program Comprehension. pp. 328–331. IEEE (2017)
10. Fan, L., Liu, J., Liu, Z., Lo, D., Xia, X., Li, S.: Exploring the capabilities of llms for code-change-related tasks. *ACM Transactions on Software Engineering and Methodology* **34**(6), 1–36 (2025)
11. Fowler, M.: *Refactoring: improving the design of existing code*. Addison-Wesley Professional (2018)
12. Gorschek, T., Garre, P., Larsson, S., Wohlin, C.: A model for technology transfer in practice. *IEEE software* **23**(6), 88–95 (2006)
13. Gupta, A., Sundaresan, N.: Intelligent code reviews using deep learning. In: Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining Deep Learning Day (2018)
14. Hu, R., Wang, X., Wen, X.C., Zhang, Z., Jiang, B., Gao, P., Peng, C., Gao, C.: Benchmarking llms for fine-grained code review with enriched context in practice. In: Proceedings of the 34th ACM International Conference on the Foundations of Software Engineering. pp. 620–631 (2026)
15. Johnson, B., Song, Y., Murphy-Hill, E., Bowdidge, R.: Why don’t software developers use static analysis tools to find bugs? In: 2013 35th International Conference on Software Engineering. pp. 672–681. IEEE (2013)
16. Khomh, F., Di Penta, M., Guéhéneuc, Y., Antoniol, G.: An exploratory study of the impact of antipatterns on software changeability. *ecole polytechnique de montréal. Tech. rep., Tech. Rep. EPM-RT-2009-02* (2009)
17. Kononenko, O., Baysal, O., Godfrey, M.W.: Code review quality: How developers see it. In: Proceedings of the 38th international conference on software engineering. pp. 1028–1038 (2016)

18. Kudrjavets, G., Kumar, A., Nagappan, N., Rastogi, A.: Mining code review data to understand waiting times between acceptance and merging: An empirical analysis. In: *Proceedings of the 19th International Conference on Mining Software Repositories*. pp. 579–590 (2022)
19. Li, H.Y., Shi, S.T., Thung, F., Huo, X., Xu, B., Li, M., Lo, D.: Deepreview: automatic code review using deep multi-instance learning. In: *Pacific-Asia Conference on Knowledge Discovery and Data Mining*. pp. 318–330. Springer (2019)
20. Nashaat, M., Miller, J.: Towards efficient fine-tuning of language models with organizational data for automated software review. *IEEE Transactions on Software Engineering* **50**(9), 2240–2253 (2024)
21. Offermann, P., Levina, O., Schönherr, M., Bub, U.: Outline of a design science research process. In: *Proceedings of the 4th International Conference on Design Science Research in Information Systems and Technology*. pp. 1–11 (2009)
22. Ramesh, S., Bose, J., Singh, H., Raghavan, A., Chowdhury, S.R., Sridhara, G., Saini, N., Britto, R.: Automated code review using large language models at ericsson: An experience report. In: *2025 IEEE International Conference on Software Maintenance and Evolution*. pp. 602–607. IEEE (2025)
23. Ren, X., Dai, C., Huang, Q., Wang, Y., Liu, C., Jiang, B.: Hydra-reviewer: A holistic multi-agent system for automatic code review comment generation. *IEEE Transactions on Software Engineering* (2025)
24. Sergeyuk, A., Lvova, O., Titov, S., Serova, A., Bagirov, F., Bryksin, T.: Assessing consensus of developers' views on code readability. *arXiv preprint arXiv:2407.03790* (2024)
25. Sergeyuk, A., Lvova, O., Titov, S., Serova, A., Bagirov, F., Kirillova, E., Bryksin, T.: Reassessing java code readability models with a human-centered approach. In: *Proceedings of the 32nd IEEE/ACM international conference on program comprehension*. pp. 225–235 (2024)
26. Shi, S.T., Li, M., Lo, D., Thung, F., Huo, X.: Automatic code review by learning the revision of source code. In: *Proceedings of the AAAI conference on artificial intelligence*. pp. 4910–4917. No. 01 (2019)
27. Smith, C.U., Williams, L.G.: Software performance antipatterns for identifying and correcting performance problems. In: *Int. CMG Conference* (2012)
28. Sun, T., Xu, J., Li, Y., Yan, Z., Zhang, G., Xie, L., Geng, L., Wang, Z., Chen, Y., Lin, Q., et al.: Bitsai-cr: Automated code review via llm in practice. In: *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering*. pp. 274–285 (2025)
29. Tantithamthavorn, K., Zou, Y., Wong, A., Gupta, M., Wang, Z., Buller, M., Jiang, R., Watson, M., Jeong, M., Chen, K., et al.: Rovodev code reviewer: A large-scale online evaluation of llm-based code review automation at atlassian. In: *Proceedings of the IEEE/ACM 48th International Conference on Software Engineering: Software Engineering in Practice*. pp. 34–45 (2026)
30. Unterkalmsteiner, M., Badampudi, D., Britto, R., Ali, N.B.: Help me to understand this commit!-a vision for contextualized code reviews. In: *Proceedings of the 1st ACM/IEEE workshop on integrated development environments*. pp. 18–23 (2024)
31. Wert, A.: *Performance problem diagnostics by systematic experimentation*, vol. 20. KIT Scientific Publishing (2018)
32. Yang, Z., Gao, C., Guo, Z., Li, Z., Liu, K., Xia, X., Zhou, Y.: A roadmap for modern code review: Challenges and opportunities. *ACM Transactions on Software Engineering and Methodology* (2026)